PRODUCT REQUIREMENT DOCUMENT (PRD)
&
SYSTEM ARCHITECTURE PROPOSAL

Explainable Ensemble Learning Framework for Intrusion Detection in Smart Campus Networks

Document Version: 2.0 (Mentor-Updated Specification)	**Target Release:** Major Project & Publication Deliverable
Team Name / Group No: Tyfonix Security+	**Domain:** Cybersecurity, IoT & Machine Learning

1. Team Roles & Ownership Structure

To maintain software engineering best practices, project responsibilities are distributed among 5 specialized engineering roles:

- **Member 1 -Priyo Chand (Tech Lead & Machine Learning Architect):** Owns overall system technical decisions, baseline ML model implementations, hyperparameter tuning, ensemble architectural choices (voting/stacking), and code review standards.
- **Member 2 – Anshuman Sahu (Explainable AI & Analytics Specialist):** Owns integration of SHAP, LIME, and feature attribution modules. Responsible for translating complex matrix outputs into human-understandable visual explanations.
- **Member 3 – Swadesh Dey (Data & Backend Integration Engineer):** Owns raw dataset ingestion, data preprocessing/cleaning pipelines, feature scaling/encoding, and API endpoint construction using Python backend frameworks.
- **Member 4 – Tanmoy Majumder (Frontend & Dashboard Developer):** Owns user interface layout, live network traffic log visualization, alert triggers, and interactive rendering of SHAP/LIME explanation plots for administrators.
- **Member 5 – Kumar Sahil (DevOps, QA & Cloud Release Engineer):** Owns Git repository policies (branching strategy, PR checks), automated unit/integration testing suites, environment containerization, and staging/cloud deployment.

2. Executive Summary

Smart campus ecosystems integrate vast networks of automated Wi-Fi access points, IoT sensors, surveillance systems, and administrative computing platforms. As network density scales, cybersecurity threats become increasingly complex. This project presents an intelligent Intrusion Detection System (IDS) designed to analyze smart campus network traffic in real time. By combining multiple ensemble machine learning models with Explainable AI (XAI) frameworks (SHAP and LIME), the system provides high attack-detection precision alongside clear explanations of why specific traffic flows are flagged as malicious. Per mentor directives (Ranadip Kundu Sir), the system follows a dual-track evaluation strategy: Edge-IIoTset serves as the primary dataset for immediate smart-campus product demonstration, UNSW-NB15 provides a baseline benchmark, and NF-UQ-NIDS-v2, CICIoT2023, and BoT-IoT enable source-aware cross-dataset generalisation and stress-testing for research publication.

3. Problem Statement

Traditional rule-based and legacy Intrusion Detection Systems (IDS) suffer from core weaknesses in modern smart campus environments:

- **1. High False-Positive Rates & Poor Generalization:** Standard detection mechanisms struggle with modern dynamic traffic patterns (e.g., DDoS, Port Scanning, Malware, Probe attacks, IoT protocol exploits), generating excessive false alarms.
- **2. The 'Black Box' Dilemma:** Advanced Machine Learning (ML) solutions often deliver high predictive accuracy but fail to explain the mathematical rationale behind their classifications. Network administrators cannot verify whether an alert is a genuine intrusion or a harmless traffic spike.
- **3. Outdated Dataset Benchmarks:** Legacy datasets like NSL-KDD no longer reflect modern IoT communication protocols (MQTT, CoAP, HTTP) or recent threat vectors, making cross-dataset generalisation critical for real-world deployments.

4. Goals and Objectives

4.1 Primary Goals

- **Ensemble Classification:** Develop an ensemble learning framework capable of accurately categorizing smart campus traffic into Normal versus Malicious attack classes.
- **Model Interpretability:** Demystify model predictions by embedding Explainable AI (SHAP & LIME) to highlight specific network features driving each alert.
- **Research Generalisation:** Validate cross-dataset model robustness across diverse IoT/network sources for high-impact publication development.

4.2 Key Objectives

- **Accuracy & Reliability:** Achieve an F1-score ≥ 0.95 with minimal false-alarm generation across multi-class IoT attack vectors.
- **Explainability:** Generate feature attribution graphs (global summary plots and local packet-level explanations) within low latency limits.
- **Operational Readiness:** Deliver a fully functional REST API and interactive administrator web dashboard for real-time traffic analysis.
- **Publication Validation:** Execute source-aware cross-dataset evaluation using NF-UQ-NIDS-v2, CICIoT2023, and BoT-IoT.

5. Target Users

- **Campus Network Administrators:** Require real-time visibility into active threats and clear, feature-level explanations to take immediate corrective security actions.
- **Cybersecurity Operations Personnel (SOC Analysts):** Need low false-positive alerts, detailed confusion metrics, and local SHAP/LIME breakdown reports for post-incident audits.
- **Academic/Project Evaluators & Mentors:** Need access to an end-to-end, reproducible system with measurable benchmarks, interactive UI demos, and cross-dataset research experiments.

6. Project Scope

6.1 In-Scope

- • Ingestion, cleaning, feature scaling, and categorical encoding of industry-standard network intrusion datasets (e.g., UNSW-NB15, CIC-IDS2018, or NSL-KDD).
- • Implementation of multiple baseline ML models and advanced Ensemble Voting/Stacking architectures.
- • Integration of SHAP (Shapley Additive exPlanations) for global feature ranking and LIME (Local Interpretable Model-agnostic Explanations) for individual attack flow explanations.
- • Development of a RESTful backend service for inference and XAI generation.
- • Development of an administrator dashboard displaying live traffic classifications, system logs, and XAI visualizations.
- • Containerized environment setup using Docker for unified local/cloud deployment.

6.2 Out-of-Scope (Future Enhancements)

- • Physical hardware deployment on live, active campus routers or optical network switches (data will be ingested via log file uploads or stream simulation).

- • Automatic automated physical response triggers (e.g., automatically executing firewall port blocks or hardware-level IP isolation).
- • Retraining models on custom raw packet captures (.pcap) without pre-extracted network flow features.

7. Dataset Strategy

As instructed by mentor Mr. Ranadip Kundu Sir, the dataset strategy is formally partitioned into Product Development and Research Publication tracks:

Dataset Name	Role in Project Strategy	Technical Rationale & Purpose
Edge-IIoTset	Primary (Product & Campus Demo)	Main IoT dataset for core app development, live Streamlit dashboard demo, and smart-campus traffic evaluation.
UNSW-NB15	Secondary Baseline	Secondary benchmark for immediate college project reporting and baseline comparison.
NF-UQ-NIDS-v2	Research Generalisation	Unified NetFlow source-aware dataset used in publication development to evaluate cross-dataset performance.
CICIoT2023	Independent IoT Validation	Recent independent IoT dataset used in publication development to test zero-day / unseen threat detection.
BoT-IoT	Stress-Test Dataset	Heavy DDoS and imbalanced dataset used specifically to stress-test system limits under severe traffic spikes.
NSL-KDD	EXCLUDED / REJECTED	Explicitly excluded from main study due to legacy, outdated 1990s traffic protocols.

8. User Stories

ID	As a...	I want to...	So that I can...
US-01	Network Admin	Upload network log files or send sample JSON packet metrics to the system	Evaluate whether traffic is Normal or an Attack.
US-02	SOC Analyst	View local XAI explanations (SHAP/LIME) for a flagged alert	Understand which specific features (e.g., Source Bytes, Packet Rate) caused the alert.
US-03	Network Admin	View global SHAP summary charts across the entire dataset	Identify campus-wide network vulnerabilities and overall key risk indicators.
US-04	Project Evaluator	Access an interactive dashboard with confusion matrices and performance metrics	Evaluate the performance gain of the Ensemble model over individual algorithms.
US-05	Research Mentor	View cross-dataset generalization metrics (NF-UQ-NIDS-v2 & CICIoT2023)	Verify system robustness for journal/conference paper publication.

9. Functional Requirements

- **FR-1 (Data Preprocessing):** System must clean missing values, normalize numerical features (e.g., Standard/MinMax Scaling), and encode categorical protocol fields for Edge-IIoTset and secondary datasets.
- **FR-2 (Ensemble Prediction):** System must execute predictions through an ensemble mechanism (Soft Voting or Stacking using Random Forest, XGBoost, and LightGBM) and output classification probabilities alongside attack labels.
- **FR-3 (XAI Computation):** System must output Global Explanation Plots illustrating overall feature impact across historical data, as well as Local Instance Explanations (SHAP Waterfall / LIME force plots) explaining single alert instances.

- **FR-4 (API Serving):** System must expose HTTP REST endpoints accepting JSON requests containing network flow metrics and returning predictions + XAI payloads.
- **FR-5 (Dashboard Display):** System must render real-time charts, traffic log history, attack distribution counters, and embedded XAI plots.
- **FR-6 (Cross-Dataset Evaluation Engine):** System must support loading independent test payloads (CICIoT2023, NF-UQ-NIDS-v2) to evaluate and log zero-day generalization accuracy.

10. Non-Functional Requirements (NFRs)

- **NFR-1 (Inference Latency):** Batch model inference must execute in under 100 ms per 100-packet payload.
- **NFR-2 (Explanation Speed):** Local LIME/SHAP explanation plots must generate within 2 seconds of an alert query.
- **NFR-3 (Classification Performance):** Target F1-Score ≥ 0.95 and Recall ≥ 0.96 for malicious categories on Edge-IIoTset.
- **NFR-4 (Portability & Isolation):** System components must run reproducibly inside Docker containers across Linux, macOS, and Windows environments.
- **NFR-5 (Code Quality & Security):** Secrets/keys must be loaded strictly via environment variables (.env). Code base must adhere to PEP8/Linting standards and maintain Git branch protection rules.

11. UI/UX Requirements

- **Dashboard Overview:** Clean layout featuring top-level KPI cards (Total Requests Analyzed, Attacks Detected, Normal Traffic %, Average Latency).
- **Traffic Stream & File Upload Portal:** CSV/JSON drag-and-drop zone alongside a simulated real-time stream log.
- **Interactive XAI Modal:** Clicking on any flagged attack log triggers a slide-out panel showing LIME/SHAP force plots and top 5 influencing features.
- **Performance & Cross-Dataset Benchmark View:** Visual confusion matrices, ROC-AUC curves, and comparative metric panels comparing Edge-IIoTset performance against UNSW-NB15 and CICIoT2023.

12. Technical Requirements, Tech Stack & Architectural Options

To provide flexibility and prevent getting locked into a single framework before mentor evaluation, candidate technologies are structured across key functional layers:

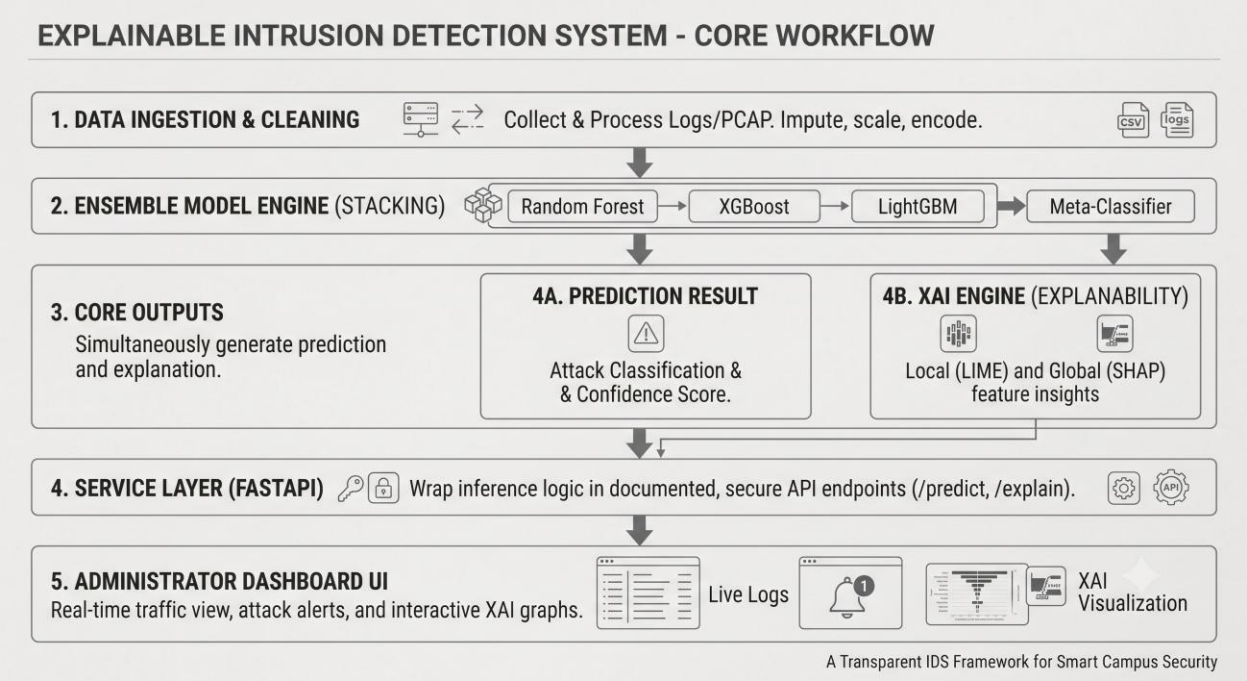
System Layer	Selected Benchmark Options	Industry Rationale
Programming Language	Primary: Python 3.10+	Standard ecosystem for ML, XAI calculations, and backend microservices.
ML Engine Options	Option A: Scikit-Learn (Random Forest) Option B: XGBoost & LightGBM Option C: Stacking Ensemble Meta-Classifier	Scikit-Learn handles standard ensembles; XGBoost/LightGBM provide high efficiency on tabular network/IoT flow logs.
XAI Frameworks	Option A: SHAP (SHapley Additive exPlanations) Option B: LIME (Local Interpretable Explanations)	SHAP provides mathematically sound global feature priorities; LIME offers fast local instance explanations.
Backend API Options	Option A: FastAPI (Async, Swagger docs) Option B: Flask (Lightweight alternative)	FastAPI offers fast async throughput and automatic OpenAPI generation.
Frontend UI Options	Option A: Streamlit (Rapid ML prototyping) Option B: React.js / Vite + Chart.js	Streamlit allows fast Python-native iteration; React delivers custom web dashboard layouts.
Containerization & Storage	Storage: PostgreSQL / SQLite / In-Memory Pandas DevOps: Docker, docker-compose, GitHub Actions	Guarantees 'works on any machine' consistency across development and evaluation environments.

13. System Architecture Blueprint

The system follows a modular 5-stage pipeline designed for low-latency network traffic analysis and model explainability, and cross-dataset testing:

```
[1. Data Ingestion (Edge-IIoTset / UNSW-NB15 / CICIoT2023)] ──► [2. Ensemble ML Engine
                                                                (RF + XGB + LGBM)]
                                                                          │
[5. Admin Dashboard UI] ◄── [4. FastAPI REST Layer] ◄── [3. XAI Engine (SHAP / LIME)]
```

Work Flow:



14. System Requirements

14.1 Development Environment Requirements

- **Operating System:** Windows 10/11, macOS, or Ubuntu Linux.
- **Processor:** Minimum 4-Core CPU (Intel i5/AMD Ryzen 5 or Apple Silicon M-Series).
- **RAM:** Minimum 8 GB (16 GB recommended for handling large CSV flow datasets in memory).
- **Disk Space:** 10 GB free space (for datasets, virtual environments, Docker images).
- **Software:** Python 3.10+, Docker Desktop, Git, VS Code / PyCharm.

14.2 Deployment Runtime Requirements

- **Runtime:** Docker Engine 20.10+ with docker-compose.
- **Hosting Options (Staging):** Render, Railway, Vercel, or cloud VM (AWS EC2 / DigitalOcean).

15. Success Metrics (KPIs)

- **Classification Accuracy:** Overall Accuracy ≥ 96% across multi-class predictions on Edge-IIoTset.
- **Attack Recall:** Recall ≥ 96% on high-risk vectors (DDoS, Ransomware, Injection) to limit false negatives.
- **False Positive Rate (FPR):** FPR ≤ 3% on benign campus traffic flows.
- **API Throughput:** REST API successfully handles ≥ 50 requests per second without timeout failures.
- **Cross-Dataset Generalisation Gain:** Maintain F1-score ≥ 0.88 when evaluated on independent test set (CICIoT2023).

16. Assumptions and Constraints

16.1 Assumptions

- • Edge-IIoTset and secondary datasets accurately reflect modern smart-campus packet structures, IoT protocols, and threat vectors.
- • Evaluation environments have Internet connectivity for pulling Docker base images and Python packages.

16.2 Constraints

- • High-density dataset files (e.g., multi-gigabyte raw pcap files) will be sampled and pre-processed into tabular flow logs to stay within academic cloud hosting limits.
- • Real-time packet parsing speed is constrained by Python CPU execution without dedicated hardware acceleration GPUs.

17. Dependencies

- **External Datasets:** Availability of open-source datasets (Edge-IIoTset, UNSW-NB15, NF-UQ-NIDS-v2, CICIoT2023, BoT-IoT).
- **Open Source Libraries:** Continuous compatibility across Python 3.10+, Scikit-Learn, XGBoost, SHAP, and FastAPI packages.
- **Infrastructure:** GitHub repository accessibility for version control and automated Actions workflows.

18. Execution Roadmap & Phase Milestones

Because this major project spans an academic year, execution is organized into 6 Logical Step Milestones:

Step Milestone 1: Requirement Scoping & Dataset Setup

Finalize primary dataset (Edge-IIoTset) and secondary baseline (UNSW-NB15), configure GitHub repository rules, assign team roles, and finalize PRD V2.

Step Milestone 2: Exploratory Data Analysis (EDA) & Preprocessing Pipeline

Perform EDA on Edge-IIoTset, address missing values, execute feature scaling/encoding, and build automated preprocessing scripts.

Step Milestone 3: Base & Ensemble Model Development

Train baseline classifiers (Random Forest, XGBoost, LightGBM), construct Stacking/Voting Meta-Classifiers, and record accuracy/F1 benchmarks.

Step Milestone 4: Explainable AI (XAI) Pipeline Integration

Integrate SHAP summary plots for global interpretability and LIME modules for local packet explanation outputs.

Step Milestone 5: API Microservice & Administrator Web Dashboard

Wrap inference logic in FastAPI endpoints and build the Streamlit/React web portal showing real-time logs and XAI visual graphs.

Step Milestone 6: Publication Experiments, Docker Containerization & Final Report

Execute cross-dataset generalization experiments (NF-UQ-NIDS-v2, CICIoT2023, BoT-IoT), write docker-compose.yml configs, compile final project report, and draft research paper.

19. Acceptance Criteria

- **AC-1:** The project repo contains a clear README.md, Dockerfile, requirements.txt, and executable code modules.
- **AC-2:** The ensemble model reliably predicts Edge-IIoTset flow features as Normal or Attack with F1 ≥ 0.95.
- **AC-3:** Every flagged attack prediction can trigger a readable XAI visualization showing top influential network features.
- **AC-4:** The web dashboard displays live inference outputs alongside performance metrics without software runtime crashes.
- **AC-5:** Cross-dataset generalization benchmarks (NF-UQ-NIDS-v2, CICIoT2023) are documented for paper submission.

20. Open Questions

- **Question 1:** Should we focus on a Binary Classification task (Normal vs Attack) or a Multi-Class Classification task (Normal, DDoS, Ransomware, Injection, PortScan) for the final presentation demo?
- **Question 2:** What specific target journal/conference (e.g., IEEE Access, Elsevier Computers & Security) is preferred for submitting the NF-UQ-NIDS-v2 cross-dataset publication paper?
- **Question 3:** Should the frontend dashboard be built using Streamlit (faster Python integration) or React.js (more custom UI design flexibility)?

21. Appendix

21.1 Glossary

- **IDS:** Intrusion Detection System
- **XAI:** Explainable Artificial Intelligence
- **SHAP:** SHapley Additive exPlanations (game-theoretic feature attribution)
- **LIME:** Local Interpretable Model-agnostic Explanations
- **PRD:** Product Requirement Document
- **NFR:** Non-Functional Requirement
- **IoT:** Internet of Things

21.2 References

- • Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset for IoT and IIoT Applications.
- • UNSW-NB15 Network Intrusion Dataset Repository.
- • NF-UQ-NIDS-v2: NetFlow-based Unified Network Intrusion Detection Dataset.
- • CICIoT2023 Dataset Repository (Canadian Institute for Cybersecurity).
- • BoT-IoT Dataset Repository.
- • SHAP & LIME Official Documentation.